Ferramentas e Tecnologias

Projeto CIFRA — Challenge Leroy Merlin

1. Visão geral

O CIFRA é composto por três aplicações web publicadas e um protótipo de jogo em desenvolvimento paralelo. As três aplicações vivem num único repositório organizado como *monorepo* — um repositório que abriga vários projetos independentes, mas que compartilham código e ferramentas.

Aplicação	O que é	Endereço
Site	Conteúdo de conscientização sobre golpes digitais	`cyber-guard.vercel.app`
Jogo	Plataforma gamificada de treinamento corporativo	`cyber-guard-jogo.vercel.app`
API	Autenticação, progresso e ranking dos jogadores	`cyber-guard-api.vercel.app`

A divisão em três projetos separados, e não um só, atende a uma diferença real de natureza: o site é conteúdo público que não guarda nada sobre quem o acessa, enquanto o jogo precisa de contas, progresso salvo e ranking. Manter os dois separados garante que o site continue funcionando mesmo que o banco de dados esteja fora do ar, e permite que cada um seja atualizado sem afetar o outro.

2. Arquitetura do repositório

Ferramenta	Versão	Para que serve
pnpm	11.18.0	Gerenciador de pacotes; organiza os projetos em *workspaces*
Turborepo	2.x	Orquestra as tarefas de build respeitando as dependências entre projetos
TypeScript	5.x	Linguagem base de todo o código; JavaScript com verificação de tipos
Node.js	22.23.0	Ambiente de execução
Git / GitHub	—	Controle de versão e origem dos deploys automáticos

O repositório tem quatro *workspaces*:

- `apps/site` — o site de conscientização
- `apps/jogo` — a plataforma gamificada
- `apps/api` — o servidor de dados
- `packages/shared` — as regras do jogo e o conteúdo das fases, usados tanto pelo jogo quanto pela API

O `packages/shared` existe para resolver um problema concreto: tanto o navegador quanto o servidor precisam saber como calcular a pontuação de uma fase. Se essa regra fosse escrita duas vezes, uma correção feita num lado deixaria o outro divergente — e a pontuação exibida na tela não bateria com a

gravada no banco. Com o código compartilhado, existe uma definição só.

O Turborepo entra como o orquestrador. O arquivo `turbo.json` declara que a tarefa `build` de um projeto depende do `build` de suas dependências (`dependsOn: ["^build"]`), de modo que o `shared` é sempre compilado antes de quem o utiliza. Ele também guarda em cache os resultados: quando nada mudou num projeto, a tarefa é reaproveitada em vez de refeita.

3. Site de conscientização

Tecnologia	Versão	Para que serve
Next.js	15.5.23	Framework React que gera as páginas e cuida do roteamento
React	19.2.8	Biblioteca de interface; monta a tela a partir de componentes
Tailwind CSS	3.4.15	Estilização por classes utilitárias direto no HTML
Framer Motion	11.18.2	Animações de entrada e transição
PostCSS + Autoprefixer	8.5 / 10.4	Processam o CSS e adicionam prefixos de compatibilidade
clsx + tailwind-merge	2.1 / 2.5	Combinam classes CSS condicionais sem conflito

O site tem doze páginas: início, `/senha`, `/golpe`, `/anatomia`, `/agora`, `/ja-cai`, `/no-trabalho`, `/ia`, `/glossario`, `/treinamento`, `/sobre` e `/contato`.

A característica técnica mais relevante é que **o site não envia nada para nenhum servidor**. A página inicial demonstra ao visitante o que o próprio navegador dele revela — resolução de tela, fuso horário, idioma, tipo de conexão — e todo esse processamento acontece localmente. As informações são lidas por interfaces padrão da web (`screen`, `window.devicePixelRatio`, `Intl.DateTimeFormat`, `window.matchMedia`, `window.localStorage`) e nunca saem da máquina do visitante.

A identidade visual foi codificada como tokens no `tailwind.config.ts`, com regras de uso documentadas no próprio arquivo: o chumbo é o fundo, o papel é o texto, a menta marca ação e conclusão, o petróleo só aparece como campo, e o âmbar — a cor de risco — é usado exclusivamente em bloco preenchido. Amarrar essas regras ao arquivo de configuração evita que o significado das cores se perca conforme novas telas são criadas.

4. Plataforma gamificada

Tecnologia	Versão	Para que serve
Next.js	16.2.12	Framework React (versão mais recente que a do site)
React	19.2.4	Biblioteca de interface
Tailwind CSS	4.x	Estilização; nesta versão a configuração vive no próprio CSS
TanStack React Query	5.62	Busca dados da API, com cache e revalidação automática
Zustand	5.0	Guarda o estado da sessão de jogo em andamento
Radix UI	1.1	Componentes de interface acessíveis por padrão
Framer Motion	11.15	Animações

Tecnologia	Versão	Para que serve
Tone.js	15.1	Síntese de áudio para a trilha sonora
Lucide React	0.469	Conjunto de ícones
Zod	3.24	Validação de dados recebidos da API
class-variance-authority	0.7	Organiza variações visuais de componentes

São quinze telas, agrupadas por função: autenticação (`/login`, `/change-password`), navegação principal (início, `/phases`, `/ranking`, `/profile`, `/shop`, `/achievements`, `/settings`, `/admin`) e jogo propriamente dito (`/play/[dayId]`, resultados, e os minijogos `/minigames/quiz` e `/minigames/caca-phishing`).

O **Radix UI** merece destaque num projeto sobre segurança e boas práticas: ele fornece componentes como diálogos, abas e tooltips já resolvidos em termos de navegação por teclado e leitores de tela. Usar componentes prontos e acessíveis é mais confiável do que reimplementar esse comportamento.

O **TanStack React Query** cuida de toda a comunicação com a API (`lib/api/`). Ele evita requisições duplicadas, mantém os dados em cache e os revalida quando necessário.

O **Zustand** guarda o estado da partida em andamento (`lib/stores/gameSessionStore.ts`) — dados que só fazem sentido enquanto o jogador está numa fase e não precisam ir ao servidor a cada clique.

O **Tone.js** (`lib/lofi-music.ts`) sintetiza a trilha sonora no próprio navegador, em vez de carregar arquivos de áudio prontos.

> *Nota sobre as versões: o site usa Next.js 15 com Tailwind 3, e o jogo usa Next.js 16 com Tailwind 4. A diferença existe porque os dois foram desenvolvidos por equipes distintas e depois integrados. Como cada projeto é compilado e publicado de forma independente, a diferença não causa conflito.*

5. API e persistência de dados

Tecnologia	Versão	Para que serve
NestJS	11.2.1	Framework de backend com arquitetura em módulos
Express	5.2.1	Servidor HTTP sobre o qual o NestJS roda
Prisma	5.22.0	ORM: traduz entre o banco de dados e o código TypeScript
PostgreSQL	—	Banco de dados relacional (hospedado no Supabase)
Zod	3.24	Valida os dados que chegam nas requisições
bcryptjs	2.4.3	Gera o hash das senhas
@nestjs/jwt	11.0.2	Emite e verifica os tokens de sessão
cookie-parser	1.4.7	Lê os cookies enviados pelo navegador
reflect-metadata	0.2.2	Suporte aos decoradores que o NestJS usa
RxJS	7.8.1	Programação reativa, usada internamente pelo NestJS

A API está organizada em seis módulos de domínio — `auth`, `players`, `phases`, `ranking`, `upgrades` e `admin` — mais os módulos de apoio `prisma`, `common` e o arquivo `config.ts`.

Autenticação

O fluxo combina três mecanismos. As senhas nunca são armazenadas: o `bcryptjs` gera um *hash*, uma transformação irreversível, e é esse valor que vai para o banco. No login, a senha digitada é transformada da mesma forma e os resultados são comparados.

A sessão é mantida por um **token JWT** entregue num cookie com três proteções: `HttpOnly` impede que JavaScript da página leia o token, o que neutraliza uma classe inteira de ataques; `Secure` exige HTTPS; e `SameSite=Lax` impede que o cookie seja enviado em requisições vindas de outros sites.

O controle de acesso é feito por *guards* — o `AuthGuard` verifica se há sessão válida, e o `RequireRole("ADMIN")` restringe as rotas administrativas.

Validação de entrada

Todos os dados recebidos passam pelo **Zod** antes de chegar à lógica de negócio, através de um `ZodValidationPipe` integrado ao NestJS. Uma requisição malformada é rejeitada na porta de entrada, com mensagem clara, em vez de causar erro no meio do processamento.

Modelo de dados

O banco tem quatro tabelas e um tipo enumerado:

Modelo	Guarda
`User`	Conta, senha em hash, papel (ADMIN/EMPLOYEE) e progresso geral
`OwnedUpgrade`	Itens que o jogador comprou
`UnlockedAchievement`	Conquistas desbloqueadas
`PhaseResult`	Resultado de cada fase concluída
`Role` (enum)	Os papéis possíveis: ADMIN e EMPLOYEE

As três tabelas de progresso apontam para o usuário com `onDelete: Cascade` — apagar uma conta remove automaticamente todo o progresso associado, sem deixar registros órfãos. O versionamento do banco é feito pelo **Prisma Migrate**, com duas migrações aplicadas até aqui.

6. Infraestrutura e publicação

Serviço	Para que serve
Vercel	Hospedagem das três aplicações, com deploy automático a cada push
Supabase	Banco PostgreSQL gerenciado
PgBouncer	Gerenciador de conexões do Supabase (*connection pooler*)

Cada aplicação é um projeto separado na Vercel, apontando para o mesmo repositório mas com uma pasta raiz diferente. Um `git push` na branch principal dispara os três deploys.

A API roda como **função serverless**: em vez de um servidor ligado o tempo todo, o código é executado sob demanda a cada requisição. Isso reduz custo, mas impõe uma restrição importante — cada execução pode abrir sua própria conexão com o banco, e um PostgreSQL comum esgotaria o limite de conexões

rapidamente.

A solução é o **PgBouncer**, que fica entre a aplicação e o banco reaproveitando conexões. Ele exige que o Prisma seja avisado de sua presença: a URL de conexão precisa dos parâmetros `pgbouncer=true` e `connection_limit=1`, sem os quais o Prisma tenta usar *prepared statements*, que o PgBouncer em modo transação não suporta. Como essa URL costuma ser colada à mão num painel, o `prisma.service.ts` detecta a porta do pooler e completa os parâmetros automaticamente.

As migrações usam uma segunda URL (`DIRECT_URL`), que aponta direto para o banco, porque alterar a estrutura de tabelas exige conexão real.

7. Ferramentas de desenvolvimento e qualidade

Ferramenta	Versão	Para que serve
Vitest	2.1	Testes automatizados das regras de jogo
tsup	8.3	Compila o pacote compartilhado
tsx	4.19	Executa TypeScript direto, sem compilar antes
ESLint	9.x	Análise estática; aponta erros e inconsistências de estilo
Prisma CLI	5.22	Gera o cliente de banco e aplica migrações

O pacote `packages/shared` tem **18 testes automatizados** distribuídos em três arquivos, cobrindo o motor de pontuação (`scoring`), a máquina de estados das fases (`phaseMachine`) e os minijogos. São justamente as regras cujo erro seria silencioso — uma pontuação calculada errado não quebra a tela, só produz um número incorreto.

O arquivo `pnpm-workspace.yaml` traz uma seção de `overrides`, que força versões corrigidas de bibliotecas indiretas com vulnerabilidades conhecidas (`postcss`, `nanoid`, `sharp`, `qs`, `body-parser`). O critério adotado foi que `pnpm audit --prod` deve reportar zero vulnerabilidades.

Scripts de operação

Foram desenvolvidos quatro utilitários próprios, na pasta `scripts/`:

Script	O que faz
`verificar-api.js`	Testa as rotas em produção e distingue erro do servidor de erro da plataforma
`testar-handler.js`	Sobe a API localmente e valida que ela inicializa, antes de publicar
`criar-usuario.js`	Cria usuário direto no banco, com o mesmo hash da aplicação
`apagar-usuario.js`	Remove usuário, com etapa de confirmação e proteção do último administrador

O `verificar-api.js` nasceu de um problema real de diagnóstico: um erro 404 pode vir do servidor ("essa rota não existe") ou da plataforma ("essa função não existe"), e os dois são visualmente idênticos. O script separa os dois casos pela origem da resposta.

8. Protótipo em Godot

Tecnologia	Versão	Para que serve
Godot Engine	4.7	Motor de jogos 2D/3D de código aberto
GDScript	—	Linguagem de script do Godot

A pasta `Jogo/` contém um protótipo de jogo em estilo RPG, com personagem que se movimenta pelo cenário, sistema de diálogo, NPC interativo e transição entre salas. São nove arquivos de cena e recurso (`player`, `professor`, `dialogue_box`, `door`, cenários e tilesets).

Este protótipo não faz parte do que está publicado. É uma linha de exploração paralela, anterior à decisão de construir a plataforma gamificada em tecnologia web. Está documentado aqui por fazer parte do percurso técnico do projeto.

Os sprites e tilesets vêm de pacotes gratuitos do CraftPix, cuja licença não permite redistribuição em repositório público — uma restrição registrada na documentação interna do projeto.

9. Decisões técnicas e aprendizados

Esta seção reúne problemas concretos enfrentados durante a integração e a publicação, e como foram resolvidos. São situações em que o comportamento observado não correspondia ao esperado, e a investigação revelou algo não óbvio sobre as ferramentas.

Decoradores e o compilador da plataforma

O NestJS depende de *decoradores* — anotações como `@Injectable()` que descrevem como as peças se encaixam — e o compilador precisa emitir metadados sobre os tipos para que a injeção de dependência funcione. A Vercel compila funções com **esbuild**, que não implementa essa emissão. O resultado era a aplicação falhar na inicialização, com erro genérico em todas as rotas.

A solução foi compilar previamente com o `tsc` (o compilador oficial do TypeScript, que emite os metadados) e apontar a função para o resultado compilado.

Roteamento de rotas com múltiplos segmentos

Após um dos deploys, rotas como `/api/ranking` funcionavam, mas `/api/auth/login` retornava 404 — sem nenhum erro no painel. A causa: a convenção de pastas da Vercel **não implementa rotas do tipo catch-all** fora do Next.js. Um arquivo nomeado `[...path].js` é interpretado como parâmetro de nome literal e casa apenas um segmento de caminho.

A correção foi delegar o roteamento à configuração de `rewrites`, que é o padrão documentado para aplicações Express. Doze das treze rotas estavam inativas, e o problema não aparecia nos logs — só medindo cada rota individualmente.

Compilação e cache de build

Uma falha de deploy apontava um módulo interno como inexistente, mesmo com a instalação de dependências concluída com sucesso. O log revelou o motivo: a plataforma havia restaurado o cache de um deploy anterior, e a instalação tornou-se uma operação vazia. Como os *scripts de ciclo de vida* não são executados nesse caminho, o pacote compartilhado nunca foi compilado.

A correção foi delegar a ordem de compilação ao Turborepo, que resolve as dependências independentemente de a instalação ter sido executada.

Configuração que falha em silêncio

Durante a verificação de segurança, dois problemas foram identificados por medição direta. O primeiro: a API respondia a requisições de **qualquer origem** com permissão de credenciais, porque a configuração de CORS usava um valor padrão permissivo quando a variável de ambiente não estava definida.

O segundo era mais sutil: o segredo que assina os tokens de sessão tinha um valor padrão de desenvolvimento presente no repositório. Sem a variável configurada, a aplicação **funcionava normalmente** — e todas as sessões estariam assinadas com um valor público.

Ambos foram corrigidos invertendo o padrão: sem configuração explícita, a API não libera origem externa alguma, e em produção recusa iniciar sem o segredo definido. O princípio adotado é que **falha visível é preferível a funcionamento incorreto silencioso** — especialmente num projeto cujo tema é segurança.

10. Quadro-resumo

Categoria	Tecnologias
Linguagens	TypeScript, JavaScript, GDScript, SQL
Front-end	Next.js 15 e 16, React 19, Tailwind CSS 3 e 4, Radix UI, Framer Motion, Lucide, Tone.js
Estado e dados	TanStack React Query, Zustand, Zod
Back-end	NestJS 11, Express 5, Prisma 5, JWT, bcryptjs
Banco de dados	PostgreSQL, PgBouncer, Prisma Migrate
Infraestrutura	Vercel (funções serverless), Supabase, GitHub
Monorepo	pnpm workspaces, Turborepo
Qualidade	Vitest (18 testes), ESLint, TypeScript
Build	tsc, tsup, tsx, PostCSS
Jogo (protótipo)	Godot Engine 4.7

Ao todo, o projeto reúne aproximadamente **45 bibliotecas e ferramentas distintas**, distribuídas em quatro workspaces, três aplicações publicadas e um protótipo.